

5.1.1 Implementasi Sistem

Pada subbab ini, implementasi sistem merupakan tahap penerjemahan rancangan *wireframe* ke dalam bentuk visual fungsional yang menjadi titik interaksi utama antara pengguna dengan sistem. Pada tahap ini, seluruh elemen desain mulai dari halaman autentikasi login, modul manajemen *master data* pelanggan dan teknisi, hingga modul operasional survei dan proyek diwujudkan dengan mengutamakan konsistensi visual serta kemudahan navigasi.

1) Implementasi Otentikasi

Implementasi autentikasi pada sistem ini berfungsi untuk memastikan bahwa hanya pengguna yang memiliki otoritas (Admin atau Manajer Proyek) yang dapat mengakses modul internal PT SHI. Proses ini melakukan validasi terhadap data yang tersimpan pada tabel *users*.

- a. *Login Controller*: Fungsi ini bertanggung jawab untuk memvalidasi kredensial yang dimasukkan pengguna. Sistem akan mengirimkan data *email* dan *password* ke bagian *service* untuk dicocokkan dengan data di basis data *shi_iscp*. Hasil implementasi sistem otentikasi: *login controller* dapat dilihat pada Gambar 5.1.

```
// AuthController.php
public function logout(Request $request)
{
    // Mengakhiri sesi pengguna yang sedang aktif
    Auth::logout();

    // Memastikan token atau session dihapus secara
    permanen
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return response()->json([
        'code' => 200,
        'message' => 'Logout berhasil, sesi telah diakhiri
    secara aman'
    ], 200);
}
```

Gambar 5.1 Implementasi Otentikasi: Login Controller

- b. *Logout Controller*: Fungsi ini digunakan untuk mengakhiri sesi pengguna secara aman. Saat fungsi ini dipanggil, sistem akan menghapus token akses yang aktif sehingga pengguna tidak dapat lagi mengakses modul internal tanpa melakukan login kembali. Hasil implementasi sistem otentikasi: *logout controller* dapat dilihat pada Gambar 5.2.

```
// AuthController.php
public function logout(Request $request)
{
    // Mengakhiri sesi pengguna yang sedang aktif
    Auth::logout();

    // Memastikan token atau session dihapus secara permanen
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return response()->json([
        'code' => 200,
        'message' => 'Logout berhasil, sesi telah diakhiri secara
aman'
    ], 200);
}
```

Gambar 5.2 Implementasi Otentikasi: Logout Controller

2) Implementasi Proses Pengelolaan Data Pelanggan

Implementasi ini mencakup penyediaan fungsi pada *Controller* untuk mengelola data master klien yang tersimpan dalam tabel pelanggan. Dalam ekosistem Laravel, proses ini memanfaatkan *Eloquent ORM* untuk berinteraksi dengan basis data secara efisien. Hasil implementasi sistem proses pengelolaan data pelanggan dapat dilihat pada Gambar 5.3.

- a. *Store* (Menambahkan Pelanggan Baru): Fungsi ini menangani perintah *INSERT* untuk merekam identitas pelanggan baru. Data seperti nama, alamat, dan nomor telepon divalidasi terlebih dahulu sebelum disimpan ke dalam tabel pelanggan.
- b. *Index* (Menampilkan Daftar Pelanggan): Fungsi ini menjalankan perintah *SELECT* untuk mengambil seluruh data klien dari basis data. Hasilnya ditampilkan pada tabel manajemen yang memuat data pelanggan mulai dari PL0001 hingga PL0013.
- c. *Update* (Memperbarui Data Pelanggan): Fungsi ini memfasilitasi perintah *UPDATE* untuk mengubah informasi kontak klien berdasarkan *id_pelanggan*. Hal ini menjamin alamat instalasi pada sistem Smart Home tetap akurat.
- d. *Destroy* (*Soft Delete*): Fungsi ini mengimplementasikan penghapusan data secara lunak (*Soft Delete*) dengan mengisi kolom *deleted_at* tanpa menghilangkan baris data secara permanen dari basis data. Hal ini bertujuan agar riwayat penugasan atau proyek lama yang berkaitan dengan pelanggan tersebut tetap memiliki referensi data yang utuh di dalam sistem.

```
// PelangganController.php
    public function store(Request $request)
    {
        // Validasi data masukan sesuai kolom tabel
        pelanggan
        $validated = $request->validate([
            'nama_pelanggan' => 'required|string',
            'alamat' => 'required|string',
            'no_telpon' => 'required',
        ]);

        // Proses Insert menggunakan Eloquent ORM
        $pelanggan = Pelanggan::create($validated);

        return response()->json([
            'code' => 201,
            'message' => 'Data pelanggan berhasil
            didaftarkan',
            'data' => $pelanggan
        ], 201);
    }
```

```

        public function index()
        {
// Mengambil semua data pelanggan secara terurut
        $data = Pelanggan::orderBy('created_at', 'desc')-
>get();
        return response()->json([
            'code' => 200,
            'message' => 'Daftar pelanggan berhasil dimuat',
            'data' => $data
        ], 200);
    }

    public function update(Request $request, $id)
    {
        $pelanggan = Pelanggan::findOrFail($id);
        $pelanggan->update($request->all());
        return response()->json([
            'code' => 200,
            'message' => 'Informasi pelanggan berhasil
diperbarui',
            'data' => $pelanggan
        ], 200);
    }

    public function destroy($id)
    {
// Mencari data pelanggan berdasarkan Primary Key
        $pelanggan = Pelanggan::findOrFail($id);

// Menjalankan fungsi delete (mengisi kolom deleted_at)
        $pelanggan->delete();

        return response()->json([
            'code' => 200,
            'message' => 'Data pelanggan berhasil dihapus
secara lunak',
        ], 200);
    }
}

```

Gambar 5.3 Implementasi Proses Pengelolaan Data Pelanggan

3) Implementasi Proses Pengelolaan Data Teknisi

Proses pengelolaan data teknisi mencakup seluruh aktivitas manajemen sumber daya manusia yang akan ditugaskan dalam proyek instalasi maupun perbaikan. Implementasi ini memastikan manajer dapat mendaftarkan teknisi baru, melihat daftar keahlian berdasarkan *shift* kerja, serta melakukan pembaruan status personel sesuai dengan data pada tabel teknisi. Hasil implementasi sistem pengelolaan data teknisi dapat dilihat pada Gambar 5.4.

- a. *Store* (Mendaftarkan Teknisi Baru): Fungsi ini digunakan untuk memasukkan data personel baru ke dalam sistem. Data yang wajib diisi meliputi *nama_teknisi*, *shift_kerja* (pagi/siang), dan *tgl_bergabung* guna keperluan administrasi operasional.
- b. *Index* (Menampilkan Daftar Teknisi): Fungsi ini menjalankan perintah *SELECT* untuk menarik seluruh informasi teknisi dari basis data. Hasilnya ditampilkan pada antarmuka manajemen untuk membantu manajer dalam memantau sebaran teknisi berdasarkan jadwal *shift* pagi maupun siang.
- c. *Update* (Memperbarui Jadwal *Shift* Teknisi): Fungsi ini memfasilitasi perubahan informasi pada data teknisi yang sudah terdaftar, seperti perubahan pembagian waktu kerja (*shift*). Pembaruan ini secara otomatis akan memperbarui kolom *updated_at* pada basis data guna menjaga akurasi rekam jejak personel.
- d. *Destroy* (*Soft Delete*): Fungsi ini mengimplementasikan mekanisme penghapusan secara lunak menggunakan *trait* *SoftDeletes* dari Laravel. Saat teknisi dihapus, sistem tidak akan menghilangkan data secara fisik dari tabel teknisi, melainkan hanya mengisi kolom *deleted_at* dengan stempel waktu saat ini. Hal ini memastikan bahwa laporan performa teknisi pada proyek-proyek lama tidak mengalami *error* karena kehilangan data referensi teknisi terkait.

```
// TeknisiController.php
    public function store(Request $request)
    {
        // Validasi input untuk memastikan data teknisi
        sesuai format
        $validated = $request->validate([
            'id_teknisi' =>
            'required|unique:teknisi',
            'nama_teknisi' =>
            'required|string|max:100',
            'shift_kerja' =>
            'required|in:pagi,siang',
            'tgl_bergabung' => 'nullable|date',
        ]);

        // Menyimpan data ke tabel teknisi melalui Eloquent
        $teknisi = Teknisi::create($validated);

        return response()->json([
            'code' => 201,
```

```

        'message' => 'Data teknisi berhasil
    ditambahkan',
        'data'    => $teknisi
    ], 201);
}

    public function index()
    {
// Mengambil data teknisi untuk ditampilkan di tabel
manajemen

        $data = Teknisi::all();

        return response()->json([
            'code'    => 200,
            'message' => 'Daftar teknisi berhasil
    dimuat',
            'data'    => $data
        ], 200);
    }

    public function update(Request $request, $id)
    {
// Mencari teknisi berdasarkan id_teknisi (Primary Key)
        $teknisi = Teknisi::findOrFail($id);

// Melakukan update data sesuai masukan dari form
        $teknisi->update($request->all());

        return response()->json([
            'code'    => 200,
            'message' => 'Data teknisi berhasil
    diperbarui',
            'data'    => $teknisi
        ], 200);
    }

    public function destroy($id)
    {
// Mencari teknisi berdasarkan id_teknisi
        $teknisi = Teknisi::findOrFail($id);

// Menjalankan soft delete (mengisi kolom deleted_at)
        $teknisi->delete();

        return response()->json([
            'code'    => 200,
            'message' => 'Data teknisi berhasil
    dinonaktifkan (Soft Delete)'
        ], 200);
    }
}

```

Gambar 5.4 Implementasi Proses Pengelolaan Data Teknisi

4) Implementasi Proses Pengelolaan Data Survei

Implementasi ini mencakup pengelolaan siklus awal pengerjaan sistem cerdas, mulai dari pencatatan kebutuhan pelanggan hingga pengaturan jadwal kunjungan teknisi ke lokasi. Proses ini sangat krusial karena hasil survei akan menentukan apakah permintaan tersebut dapat dilanjutkan menjadi sebuah proyek aktif atau tidak. Hasil implementasi sistem pengelolaan data survei dapat dilihat pada Gambar 5.5.

- a. *Store* (Mencatat Permintaan dan Jadwal Survei): Fungsi ini menangani perintah *INSERT* untuk merekam data permintaan survei sekaligus menetapkan jadwal kunjungannya. Sistem akan menyimpan detail pekerjaan yang diminta pelanggan dan menghubungkannya dengan teknisi yang tersedia pada waktu tertentu.
- b. *Index* (Menampilkan Dashboard Jadwal Survei): Fungsi ini menjalankan perintah *SELECT* dengan relasi (*JOIN*) untuk menyajikan data secara komprehensif pada dasbor manajemen penjadwalan. Informasi yang ditampilkan mencakup *ID Survei*, *nama pelanggan*, *nama teknisi*, serta waktu pengerjaan sebagaimana terlihat pada daftar jadwal JS0067 hingga JS0069.
- c. *Update* (Memperbarui Status Jadwal dan Survei): Fungsi ini digunakan untuk memperbarui status pengerjaan survei di lapangan (seperti mengubah status menjadi "*Completed*" atau "*Cancelled*"). Pembaruan status ini akan secara otomatis mengubah indikator progres pada kartu informasi dasbor, seperti metrik "*Selesai*" atau "*Dibatalkan*".
- d. *Destroy* (*Soft Delete*): Fungsi ini mengimplementasikan penghapusan lunak pada tabel *jadwal_survei* dan *detail_survei*. Dengan mengisi kolom *deleted_at*, data survei tidak akan muncul di laporan harian, namun tetap tersimpan dalam database untuk keperluan audit manajemen jika di kemudian hari diperlukan analisis mengenai alasan pembatalan survei.

```
// SurveiController.php
    public function store(Request $request)
    {
// Validasi input untuk detail dan jadwal survei
        $validated = $request->validate([
```

```

        'id_pelanggan'      =>
        'required|exists:pelanggan,id_pelanggan',
        'id_teknisi'        =>
        'required|exists:teknisi,id_teknisi',
        'detail_pekerjaan' => 'required|string',
        'jenis_pekerjaan'  => 'required',
        'jadwal_survei'    =>
        'required|date_format:Y-m-d H:i:s',
    ]);

// 1. Simpan ke tabel detail_survei
    $detail = DetailSurvei::create([
        'id_detail_survei' => 'FS' . rand(1000,
9999),
        'id_pelanggan'    =>
$validated['id_pelanggan'],
        'detail_pekerjaan' =>
$validated['detail_pekerjaan'],
        'jenis_pekerjaan' =>
$validated['jenis_pekerjaan'],
        'status_survei'    => 'Pending'
    ]);

// 2. Simpan ke tabel jadwal_survei
    JadwalSurvei::create([
        'id_jadwal_survei' => 'JS' . rand(1000,
9999),
        'id_detail_survei' => $detail->id_detail_survei,
        'id_pelanggan'    =>
$validated['id_pelanggan'],
        'id_teknisi'       =>
$validated['id_teknisi'],
        'jadwal_survei'    =>
$validated['jadwal_survei'],
        'status_jadwal'    => 'Scheduled'
    ]);

    return response()->json(['code' => 201,
        'message' => 'Jadwal survei berhasil dibuat'], 201);
}

    public function index()
    {

// Mengambil data jadwal beserta relasi pelanggan dan
teknisi
        $data = JadwalSurvei::with(['pelanggan',
            'teknisi']->get();

```



```

        return response()->json([
            'code'      => 200,
            'message' => 'Data penjadwalan survei
berhasil dimuat',
            'data'      => $data
        ], 200);
    }

    public function updateStatus(Request $request, $id)
    {
        $jadwal = JadwalSurvei::findOrFail($id);

        // Update status jadwal (misal: dari Scheduled ke Completed)
        $jadwal->update(['status_jadwal' => $request-
>status]);

        return response()->json([
            'code'      => 200,
            'message' => 'Status jadwal survei berhasil
diperbarui',
            'data'      => $jadwal
        ], 200);
    }

    public function destroy($id)
    {
        // Melakukan soft delete pada data jadwal
        $jadwal = JadwalSurvei::findOrFail($id);
        $jadwal->delete();

        return response()->json([
            'code'      => 200,
            'message' => 'Data survei telah dihapus
secara lunak dari daftar aktif'
        ], 200);
    }
}

```

Gambar 5.5 Implementasi Proses Pengelolaan Data Survei

5) Implementasi Proses Pengelolaan Data Proyek

Implementasi ini merupakan inti dari sistem manajemen di PT SHI, di mana seluruh data instalasi dan monitoring dikendalikan secara terpadu. Proses ini mencakup konversi survei berstatus ACC menjadi proyek baru, penetapan tim teknis, hingga pemantauan siklus hidup proyek (*project cycle*). Hasil implementasi sistem pengelolaan data proyek dapat dilihat pada Gambar 5.6.

- a. *Store* (Inisialisasi Proyek dari Hasil Survei): Fungsi ini menangani perintah *INSERT* untuk membuat entitas proyek baru segera setelah survei pelanggan disetujui. Sistem secara otomatis menarik informasi dari tabel *detail_survei* untuk mengisi data awal proyek guna menjamin konsistensi informasi antara permintaan pelanggan dan rencana eksekusi.
- b. *Index* (*Dashboard Monitoring Proyek*): Fungsi ini menjalankan perintah *SELECT* untuk menyajikan visualisasi performa proyek bulanan pada dasbor. Informasi ini mencakup metrik proyek yang sedang diproses, selesai, maupun ditunda, serta daftar proyek aktif.
- c. *Show* (*Detail dan Siklus Pengerjaan Proyek*): Fungsi ini digunakan untuk menampilkan informasi mendalam mengenai satu proyek tertentu, mencakup detail area instalasi, manajer proyek yang bertanggung jawab, hingga lini masa pengerjaan (*Project Cycle*).
- d. *Destroy* (*Soft Delete*): Fungsi ini mengimplementasikan penghapusan secara lunak dengan mengisi kolom *deleted_at* pada tabel proyek. Meskipun proyek dihapus dari daftar aktif, data rincian teknis pada tabel *detail_proyek* dan alokasi tim pada *tim_teknisi* tetap terjaga integritasnya untuk kebutuhan laporan tahunan atau audit internal.

```
// ProyekController.php
    public function store(Request $request)
    {
// Validasi data proyek berdasarkan referensi survei yang disetujui
        $validated = $request->validate([
            'id_pelanggan' =>
            'required|exists:pelanggan,id_pelanggan',
            'id_detail_survei' =>
            'required|unique:proyek,id_detail_survei',
            'nama_proyek' => 'required|string',
            'area_instalasi' => 'required|string',
        ]);

// Insert ke tabel proyek dengan status default MENUNGGU JADWAL
        $proyek = Proyek::create([
            'id_proyek' => 'PJ' . rand(1000, 9999),
```

```

        'id_pelanggan'      => $validated['id_pelanggan'],
        'id_detail_survei'  =>
$validated['id_detail_survei'],
        'id_user'          => Auth::id(), // Manajer yang
membuat proyek
        'nama_proyek'      => $validated['nama_proyek'],
        'status_pekerjaan' => 'MENUNGGU JADWAL',
        'area_instalasi'   =>
$validated['area_instalasi'],
        'is_approved'      => 0
    });

    return response()->json(['code' => 201, 'message' =>
'Proyek baru berhasil diinisialisasi'], 201);
}

    public function index()
    {
// Mengambil data proyek beserta relasi pelanggan dan tim teknisi
        $proyekAktif = Proyek::with(['pelanggan',
'tim_teknisi.teknisi'])
                                ->whereNotIn('status_pekerjaan',
['BATAL', 'SELESAI'])
                                ->get();

        return response()->json([
            'code'      => 200,
            'message' => 'Data monitoring proyek berhasil
dimuat',
            'data'      => $proyekAktif
        ], 200);
    }

    public function show($id)
    {
// Mengambil detail proyek lengkap dengan histori pengerjaan
(Project Cycle)
        $detail = Proyek::with(['pelanggan', 'user',
'detail_proyek', 'jadwal_proyek'])

```

```

->findOrFail($id);

return response()->json([
    'code'      => 200,
    'message' => 'Detail siklus pengerjaan proyek
berhasil dimuat',
    'data'      => $detail
], 200);
}

public function destroy($id)
{
// Mencari data proyek berdasarkan id_proyek
$proyek = Proyek::findOrFail($id);

// Melakukan soft delete
$proyek->delete();

return response()->json([
    'code'      => 200,
    'message' => 'Data proyek berhasil dihapus secara
lunak dari sistem'
], 200);
}

```

Gambar 5.6 Implementasi Pengelolaan Data Proyek

6) Implementasi Proses Pengelolaan Data Penjadwalan Proyek

Implementasi ini merupakan tahap krusial dalam siklus manajemen proyek di PT Smart Home Inovasi untuk mengatur koordinasi waktu antara tim teknisi dan ketersediaan pelanggan di lokasi. Proses ini mengelola data pada tabel `jadwal_proyek` yang mengintegrasikan informasi rincian pekerjaan dengan penetapan tanggal serta waktu pelaksanaan secara spesifik.

- a. *Store* (Menetapkan Jadwal Pengerjaan Baru): Fungsi ini menangani perintah *INSERT* untuk mendaftarkan jadwal eksekusi proyek ke dalam sistem. Data yang

dimasukkan mencakup referensi ke *id_proyek* dan *id_detail_proyek* tertentu, serta penetapan *tgl_pengerjaan* guna memastikan kesiapan operasional lapangan.

- b. *Index (Dashboard Daftar Penjadwalan)*: Fungsi ini menjalankan perintah *SELECT* untuk menarik data jadwal aktif guna ditampilkan pada antarmuka manajemen penjadwalan.
- c. *Update (Penyesuaian Jadwal Progres)*: Fungsi ini memfasilitasi perintah *UPDATE* untuk melakukan perubahan pada rencana waktu pengerjaan atau memperbarui status progres proyek secara berkala. Hal ini memungkinkan manajer proyek untuk merespons perubahan kondisi di lapangan dengan cepat guna menjaga efisiensi mobilitas teknisi.
- d. *Destroy (Soft Delete)*: Fungsi ini mengimplementasikan penghapusan lunak dengan mengisi kolom *deleted_at* pada tabel *jadwal_proyek*. Hal ini menjamin bahwa jika sebuah jadwal pengerjaan dibatalkan, data tersebut tidak akan hilang secara fisik sehingga manajemen tetap memiliki catatan mengenai riwayat pembatalan tersebut.

```
// JadwalProyekController.php

    public function index()
    {
// Mengambil semua jadwal proyek aktif beserta relasi detail
pekerjaannya

        $data = JadwalProyek::with(['proyek',
'detail_proyek'])->get();

        return response()->json([
            'code'      => 200,
            'message' => 'Seluruh jadwal pengerjaan berhasil
dimuat',
            'data'      => $data
        ], 200);
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'id_proyek'      =>
'required|exists:proyek,id_proyek',
```

```

        'id_detail_proyek' =>
        'required|exists:detail_proyek,id_detail_proyek',
        'tgl_pengerjaan' => 'required|date',
        'waktu_pengerjaan' => 'required',
    ]);

    $jadwal = JadwalProyek::create([
        'id_jadwal_proyek' => 'JP' . rand(100, 999),
        'id_proyek' => $validated['id_proyek'],
        'id_detail_proyek' =>
    $validated['id_detail_proyek'],
        'tgl_pengerjaan' =>
    $validated['tgl_pengerjaan'],
        'waktu_pengerjaan' =>
    $validated['waktu_pengerjaan'],
        'status_progres' => 'Belum Mulai'
    ]);

    return response()->json(['code' => 201, 'message' =>
    'Penjadwalan berhasil disimpan'], 201);
}

public function update(Request $request, $id)
{
    $jadwal = JadwalProyek::findOrFail($id);
    $jadwal->update($request->all());

    return response()->json([
        'code' => 200,
        'message' => 'Progres penjadwalan berhasil
diperbarui',
        'data' => $jadwal
    ], 200);
}

public function destroy($id)
{
    // Mencari jadwal proyek berdasarkan id_jadwal_proyek
    $jadwal = JadwalProyek::findOrFail($id);

```

```
// Melakukan soft delete
    $jadwal->delete();

    return response()->json([
        'code'      => 200,
        'message' => 'Data penjadwalan berhasil dihapus
secara lunak'
    ], 200);
}
```

Gambar 5.7 Implementasi Proses Data Penjadwalan Proyek